Pachira Litepaper

Abstract

Decentralized Liquidity Management systems are services that offer a range of decentralized financial (DeFi) services under one framework. The Pachira system performs various traditional finance (TradFi) operations on-chain within a unified ecosystem, including (1) a yield-bearing exchange-traded fund, (2) treasury asset issuance, and (3) multi-asset exchange. This is achieved by utilizing a collection of DeFi primitives, including a novel one called the Liquidity Tree. We demonstrate how Liquidity Trees address an inefficiency issue not well-defined in DeFi, which we term the Stagnant Liquidity problem. Additionally, we discuss how the *CHIR* token leverages Liquidity Trees to address this inefficiency issue.

Introduction

Decentralized Finance (DeFi) is an emerging technology that caters to financial products built on secure distributed ledgers on public blockchains, with a current market cap of $65 billion USD (as of Nov 2023). DeFi is facilitated through the use of smart contracts and began its inception in 2015 with the launch of the Ethereum blockchain. Since then, there have been many notable developments in the realm of smart contracts. The launch of MakerDAO on the Ethereum mainnet in 2017 marked the first time that loans could be issued using this technology without having to go through a centralized agency. In 2018, Compound Finance introduced a protocol that allowed for the borrowing and lending of assets in a permissionless manner. In that same year, Uniswap introduced the first automated market maker (AMM) used by decentralized exchanges (DEXs), marking a significant development in the DeFi space

While not very well defined, decentralized liquidity management systems have begun to emerge, utilizing various combinations of the aforementioned technologies. In the Pachira design, we have encompassed several liquidity management services operating under one unified ecosystem: (1) a yield-bearing exchange-traded fund; (2) treasury asset issuance; and (3) a multi-asset exchange. While some projects claim to offer various other liquidity management services, they are all handled at the user interface (UI) level. In contrast, Pachira offers its services trustlessly at the contract level and does not overlap with the offerings of other liquidity management services. Thus, there are no comparables to the Pachira system, as it resides fully within its own class with the introduction and utility of the novel Liquidity Tree primitive.

In this litepaper, we first cover the constant product trading indexing and settlement problems. In the next section, we discuss how these are necessary components for addressing the Stagnant Liquidity problem through the Liquidity Tree primitive. Next, we discuss the Pachira simulator and use it to test the conjectures covered in the previous sections. Following that, we utilize these concepts to discuss the Pachira design. Finally, we summarize how the *CHIR* token is utilized within the Pachira system.

CPT Indexing and Settlement Problems

An automated market maker (AMM) protocol is the mechanism used by decentralized exchanges (DEXs) and was first introduced by Uniswap, which launched on the Ethereum mainnet in November 2018. These DEXs consist of liquidity pools (LPs) represented by various trading pairs (e.g., *ETH-USDC*, *ETH-WBTC*, etc.), acting as the AMM. Trading activity within these LPs is governed via smart contracts through the constant product trading (CPT), which is defined as:

$$(x - \Delta x)(y - \gamma \Delta y) = L^2$$

Where x is reserve0, y is reserve1, L is total supply, γ is the fee, and Δx, Δy, are swap values. In this section, we cover a simple but relevant problem that is critical to the Pachira design, which we term the ***CPT Indexing Problem*** which is this: *given a position ΔL what is the indexed value in one of the two pairing assets (ie, x or y)*.

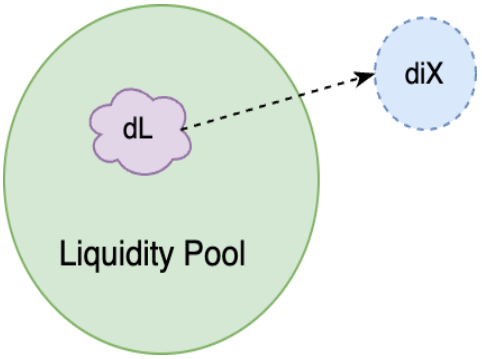

Fig 1: ***CPT Indexing Problem:*** given a position ΔL, what is the indexed value in **only** one of the two pairing assets

At first glance, the naive approach to this problem is quite simple; using the CPT formula (ie, $xy = L^2$), take the derivative of x (with respect to L) and substituting (ie, $y = L^2/x$), Δx approximates to:

$$\Delta x \sim \frac{2x\Delta L}{L}$$

Likewise, ΔL approximates to:

$$\Delta L \sim \frac{L\Delta x}{2x}$$

However, these approximations only work for small values if Δx and ΔL. In Uniswap's v3 whitepaper, they go into great detail on how to mitigate this approximation issue. However, as noted in the paper, these are based on virtual reserve values. In our problem, we are interested in attaining *Δx* and *Δy* such that we get the *true* indexed value of *ΔL*, so that every deposit is accounted for in the event every liquidity provider wants to unwind their positions in the LP. To address this problem, we must first understand that to withdraw a single asset in CPT LP, we must first perform a withdrawal to acquire both assets, then perform a swap to only leave with either one or the other asset. We solve this, by combining these two steps (ie, withdrawal and swap) into one atomic operation using this system of equations:

$$\begin{aligned}
\Delta x &= \tfrac{\Delta L x}{L}, \\
\Delta y &= \tfrac{\Delta L y}{L}, \\
\Delta y_{(i)} &= \Delta y + \tfrac{\gamma \Delta x (y - \Delta y)}{(x - \Delta x) + \gamma \Delta x}
\end{aligned}$$

where $\Delta y_{(i)}$ is the indexed token representing ΔL. This system describes the mathematical operations that are effectively being done in the contract code when performing this two-step withdrawal operation. However, our system is set up in an efficient manner where the exact requested amounts are returned to the liquidity provider. This is what we call a *WithdrawSwap*; see Moore [1] for a full detailed walkthrough on this. Using this approach assures that all assets are properly accounted for which can be used to effectively *query the LP* for how much *Δy* or *Δx* can be withdrawn any given *ΔL*. This is as if one were to perform a withdrawal of *ΔL*, followed by a swap to get only one of *Δy* or *Δx*, without having to actually withdraw the liquidity. Hence, indexed assets are determined via:

$$\begin{aligned}
f_{i,x} &: \Delta L, LP \to \Delta i x, \\
f_{i,y} &: \Delta L, LP \to \Delta i y
\end{aligned}$$

We call this approach the *indexed approach*; for more detail, see Moore [2] for a full detailed walkthrough of this mechanism. Likewise, we can also pose the question to the indexing problem in reverse, which we call the CPT Settlement Problem, which is this: *position Δx, what is the settlement value ΔL.*

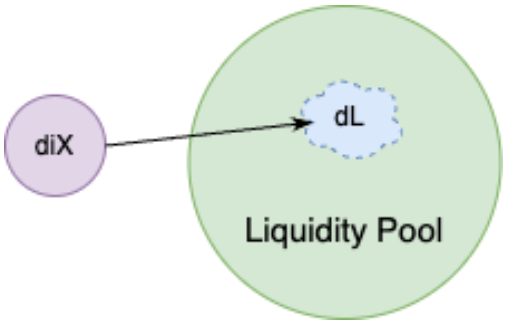

Fig 2: ***CPT Settlement Problem:*** given *Δx, what is the settlement value ΔL*

As you can see, our system (shown above) has three equations and three unknowns, thus forming a consistent system, hence, solving for ΔL is trivial. This is done plugging the first two equations of the system into the last to achieve the following:

$$\Delta L^2 \left(\frac{xy}{L^2}\right) - \Delta L \left(\frac{1000\Delta y_{(i)}x - 997\Delta y_{(i)}x + 1000xy + 997xy}{1000L}\right) + \Delta y_{(i)}x = 0$$

which is solvable in quadratic form. Thus, settlements in ΔL are determined via:

$$f_{s,x} : \Delta x, LP \to \Delta L,$$
$$f_{s,y} : \Delta y, LP \to \Delta L$$

Liquidity Trees and the Stagnant Liquidity Problem

In this section, we introduce a novel DeFi primitive which we call Liquidity (DEX) Trees. These systems serve as an extension to DEX LPs and address the problem of increasing LP capital efficiency in a trustless manner. For instance, let's consider an LP with a market cap of $100,000 USD, where only 2-5% is seen in daily trading volume. Consequently, 95-98% of that capital remains stagnant on any given day and does not earn a return on trading fees. Liquidity Trees address this issue by taking liquidity in those pools and converting them into continuous rebasing index tokens (CRITs) [3]. Sub-markets are created out of these CRITs, thus addressing what we term the Stagnant Liquidity inefficiency problem. These sub-markets are called child LPs, and the base LP from which these sub-markets are created is called the parent. This relationship is repeatable, where new CRITs can be created from the liquidity in these

child LPs to form new sub-markets, thus creating a second layer of new child markets. This repeatable structure of transitioning indexed capital between these DEXes is what we call a Liquidity Tree.

Stagnant Liquidity Problem: Include New Dimension

In the simple trading example outlined in the previous section, we discussed the issue of stagnant liquidity in an LP, which we referred to as the Stagnant Liquidity problem. Hence, the question we address is how to increase the trading volume on this stagnated liquidity in an LP over some given time interval. We achieve this by indexing the LP tokens (i.e., *ΔL*) into one of the two assets (i.e., *ix* or *iy*). We defined this process as the CPT indexing problem, as covered in a previous section. Next, we place the indexed asset (i.e., *ix* or *iy*) on the market paired against one of the two original assets (i.e., *x* or *y*). This action can be represented as a simple computational graph or by including a new dimension on our *xy* chart; see Fig. 3 for illustration in R3. This structure is called the *Simple Tree*.

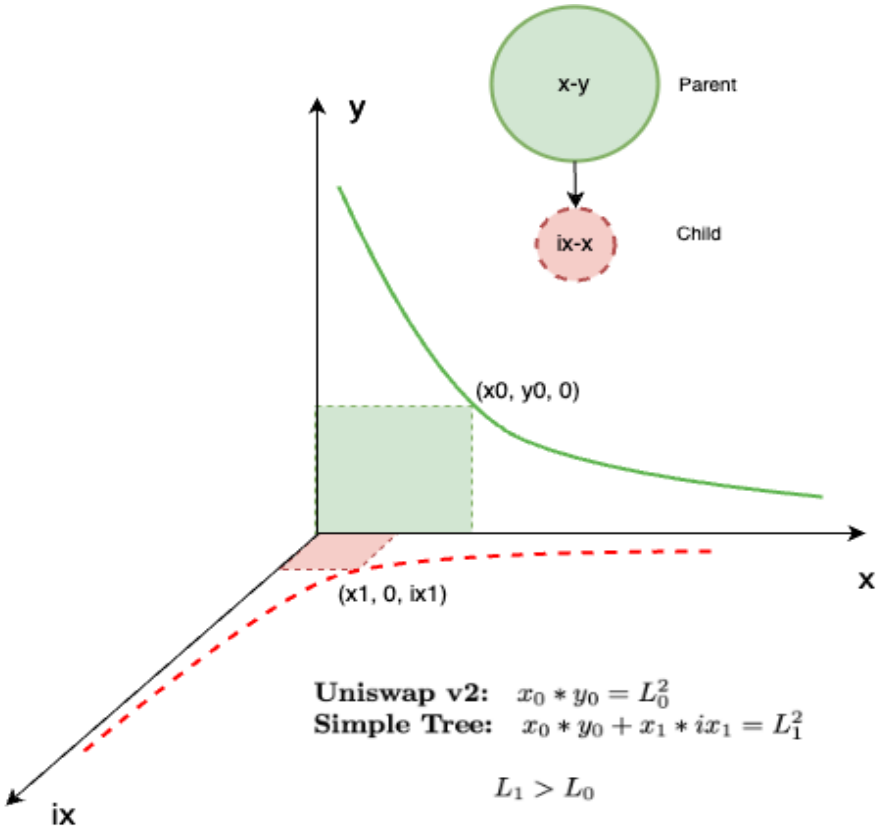

Fig 3: Include a new dimension (*ix*); liquidity *ΔL* gets indexed to *Δix*, and a new market can be created on *ix-x* plane; this is what we call a *Simple Tree*

When we place this indexed liquidity on the market, it is now effectively working in two markets (both the parent and the child market). Thus, stagnant liquidity in the parent LP that would have otherwise remained inactive in a protocol like Uniswap v2 is now exposed to the market through the child LP, collecting the standard 0.3% trading fees using the Simple Tree structure described in Fig. 3. For an illustration of the beneficial leveraging effect of the *Simple Tree* structure, refer to Fig. 4.

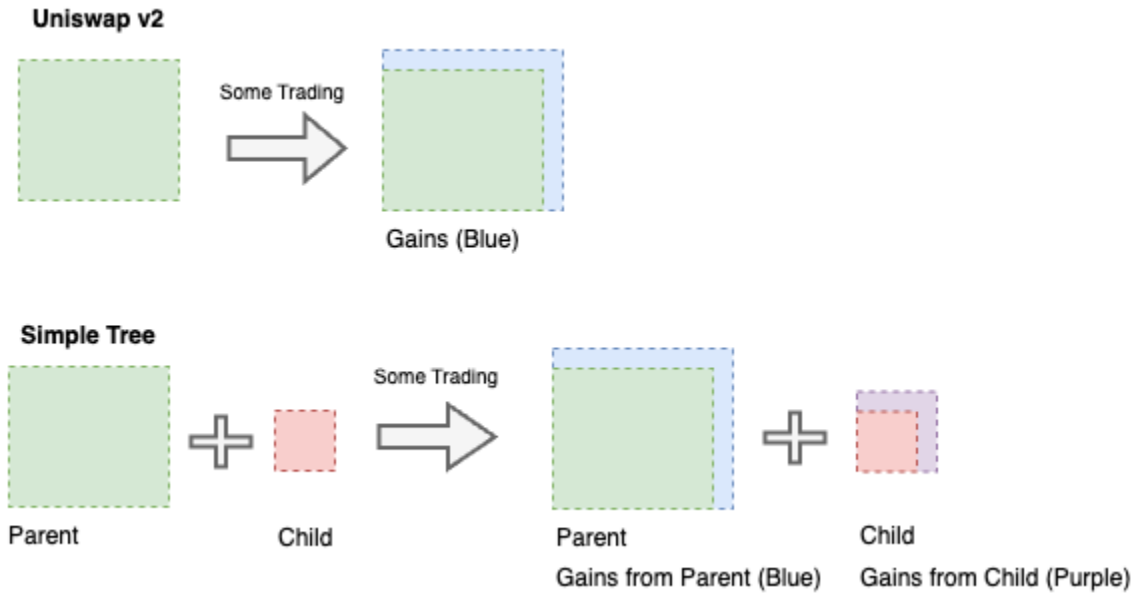

Fig 4: Boxes represent liquidity under CPT curve; creating a new market out of indexed liquidity is a way to address the Stagnant Liquidity problem; in context of coloured shaded regions, we've leveraged some of the green, received some red and made some extra purple

Liquidity Trees

If we expand on the *Simple Tree* idea discussed in the previous section, Liquidity Trees can be represented as computational graphs where nodes denote DEX operations and arcs represent indexed capital transitioning between the parent node and the child. Geometrically, this can also be described with the inclusion of additional dimensions in R^n, where these new dimensions represent indexed assets. Since there are various types of DEXes utilized in DeFi (e.g., constant weighted product, composable stable, etc.), for the sake of scope, we will only cover the CPT DEX in this section.

In the preceding section, we addressed the CPT indexing problem: given a position ΔL, what is the indexed value in only one of the two pairing assets, x or y, giving us either ix or iy. Hence, for a parent LP of assets x and y, the maximum combinations of child

markets that can be formed are *ix-y*, *ix-x, ix-iy, x-iy,* and *y-iy*. A full liquidity tree is when all possible asset combinations are utilized, forming the maximum number of child LPs under the parent; refer to Fig. 5 for a visual representation.

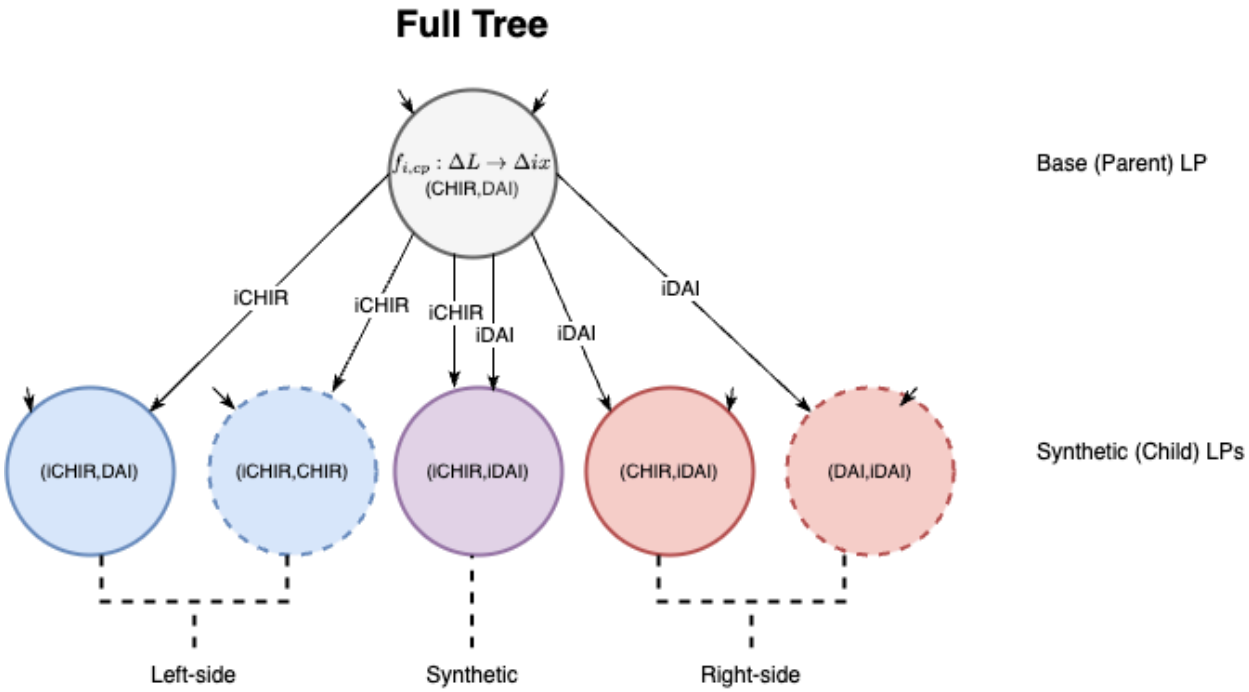

Fig 5: Full CPT liquidity tree represented as a computational tree structure comprised of left-sided, right-sided and synthetic pools

Using other asset combinations, derivations of the full tree can also be created into various sub-trees; see Fig 6.

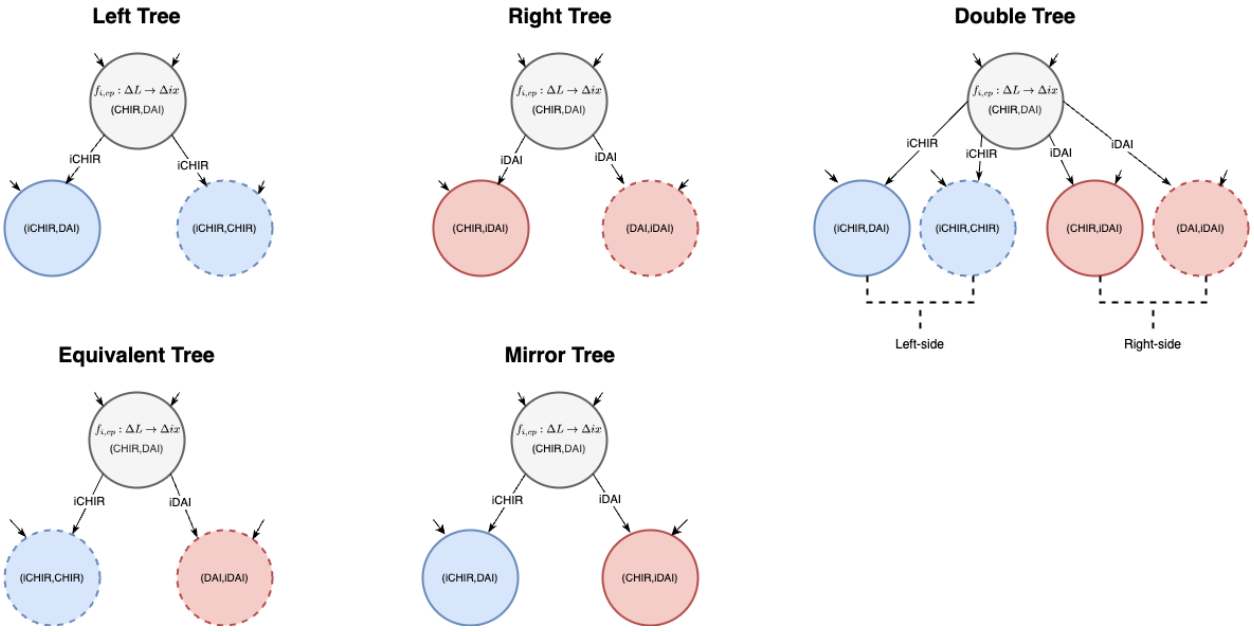

Fig 6: Sub-trees comprised of: (TOP LEFT) left-tree; (TOP CENTER) right-tree; (TOP RIGHT) double-tree; (BOTTOM LEFT) equivalent-tree; (BOTTOM CENTER) opposing-tree

If we refer to Fig. 7, the simulation of the tree (covered in the next section) clearly shows that the amount of ETH in the parent LP comfortably exceeds the aggregate amounts of ETH in the children (i.e., LP2, LP4, and LP5) over the 3+ year simulation period. This demonstrates a simulated instance where the LP within these child LPs is always fully redeemable.

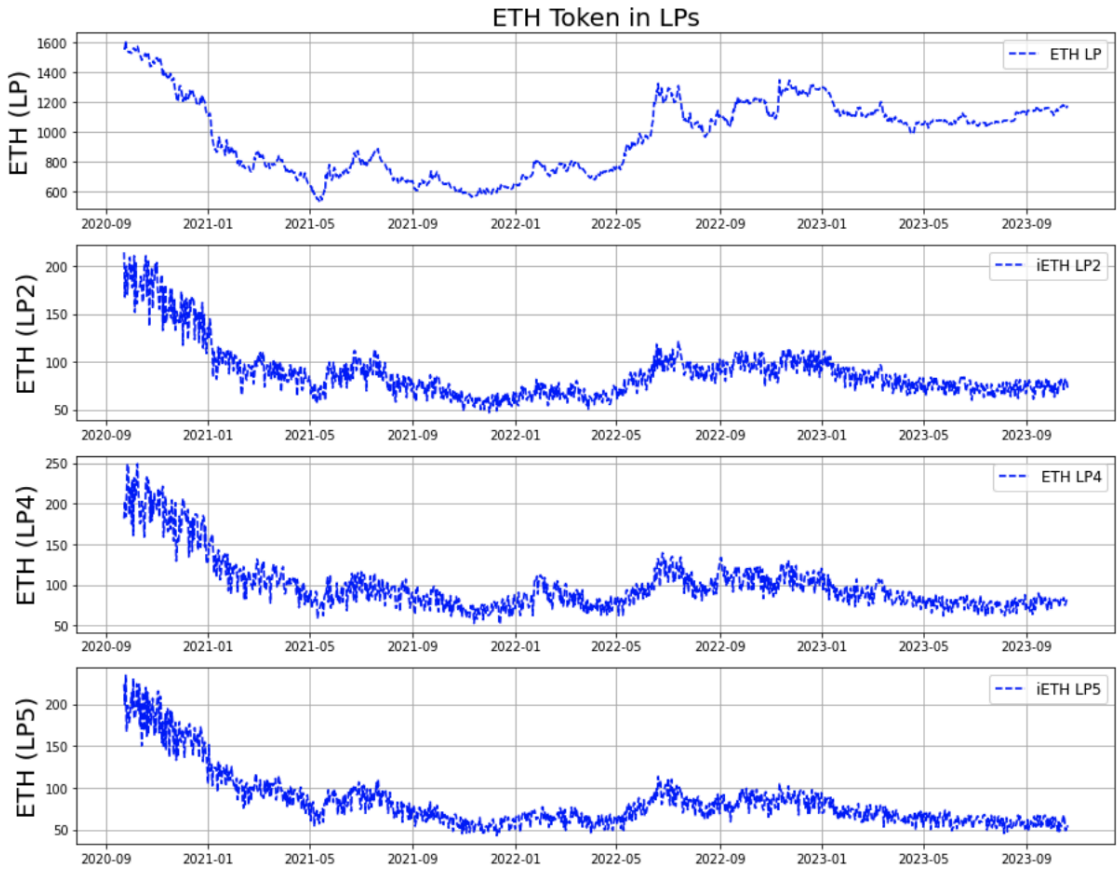

Fig 7: Native ETH amounts vs indexed ETH amounts; indexed ETH in child LPs (ie, LP2, LP4, and LP5) are fully redeemable through the parent LP

Pachira Simulator

To achieve a proper tokenomics design, it is highly inefficient to invest resources in development without first simulating the design to test specifications for various outcomes. This is a step that many DeFi projects in the crypto space overlook. Therefore, we are actively working on an open-source Python package to simulate various sandboxed DeFi components of Pachira. This tool will allow project engineers, managers, and designers to pre-plan outcomes before investing valuable resources in development.

With this tool, Pachira designers can use the plug-and-play components of our simulator to build tokenomics mockups for business planning purposes. This enables teams to gather and achieve collective consensus with potential users and investors. Moreover, this tool is not limited to the Pachira system, as it serves as a general-purpose tool to simulate DEX activity. It is beneficial for anyone wishing to set up their own LP to stress-test various ideas before committing to development. This serves as a powerful

design tool to explore the limitations of a DeFi project idea before incurring development costs (i.e., devs and project managers).

In terms of design, we utilized our simulator at every step along the design path of the Pachira system. This approach exposes DeFi to a scientific way of design, testing, and implementation. The final goal is to bring this system to a level of maturity so that it can be utilized in parallel with the contracts in real-time.

One of the initial questions we had regarding the Pachira system was, since the index tokens are unique to this system, what is the behavior in the event of an undersupply of index tokens under various edge-case scenarios? Also, would we have enough index tokens to facilitate services most of the time? To address these questions, we implemented a finite state machine, as shown in Fig. 8. This allowed us to monitor index token distribution at any given time in the simulation. We tested this distribution using Ethereum price data from January 2018 through October 2023.

Another challenge we encountered was keeping the reserve levels in the parent pool aligned with market prices. This required solving a system of nonlinear equations that gets updated at every price change in the market, ensuring that the reserve levels in the parent LP accurately reflect the market. This problem was addressed in Moore [4], and we implemented it in our simulator. In Fig. 9, we observe the result of our solution, as the LP price of *ETH-DAI* closely tracks the market price of *ETH-USD*.

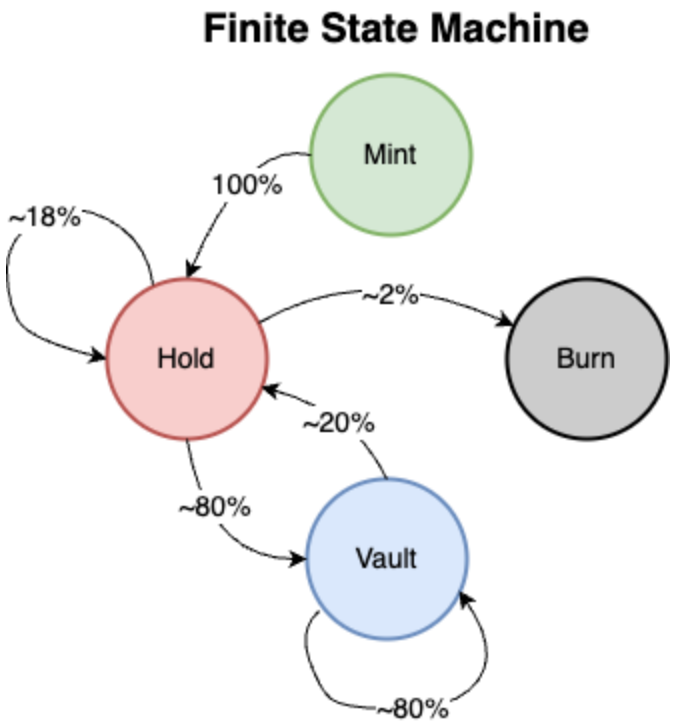

Fig 8: Finite state machine used to determine distribution of index token for tree simulation; there are four states an index token can go through: (HOLD) where index tokens are held in non-custodial wallets; (MINT) issuance of new index token out of LP token; (VAULT) index token is being held within LP; and (BURN) index token is burned in exchange for its respective native token

The significant question was how the Liquidity Tree's performance in the children compares to that of the parent. We systematically addressed this question by testing the outcomes of the full Liquidity Tree shown in Fig. 5, along with all of its subtrees in Fig. 6. The results of all these experiments are freely available for review and testing on the Syscoin GitHub repository; refer to [5].

In Fig. 8, we can observe how the child LPs (i.e., LP1-LP5) compare to the parent LP, tested on Ethereum price data from January 2018 through October 2023 for the full tree scenario in Fig. 5. This time period allowed us to witness all the various stages of the crypto business cycle. Interestingly, we observe a bifurcation pattern, presenting different investment opportunities at various stages within the cycle. These opportunities suit different objectives. For instance, if your goal is to acquire ETH, then one would invest in the *iETH-DAI* and *DAI-iDAI* LPs during the bear market. On the other hand, if your objective is to acquire USD, then one would invest in *ETH-iDAI* and *ETH-iETH* LPs during the bull market. Both of these pools significantly outperform the parent LP, which is a standard CPT LP.

Since our current focus is on design, we are not yet presenting potential strategies. However, it is our intention to do so in the near future, allowing the public to use our simulation tools as an aid for their investment strategies.

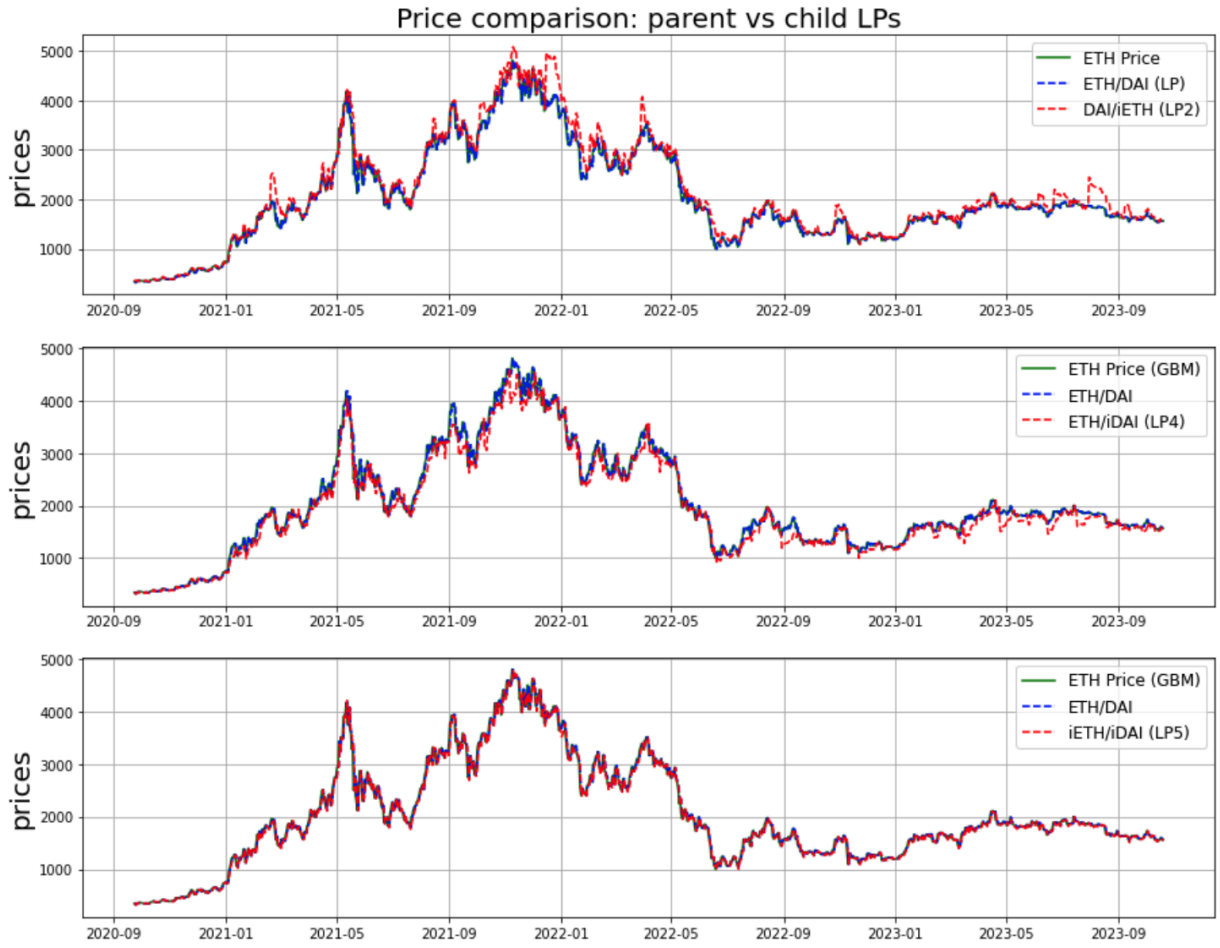

Fig 9: Trading activity in parent LP vs. trading activity in child LPs (ie, LP2, LP4, and LP5)

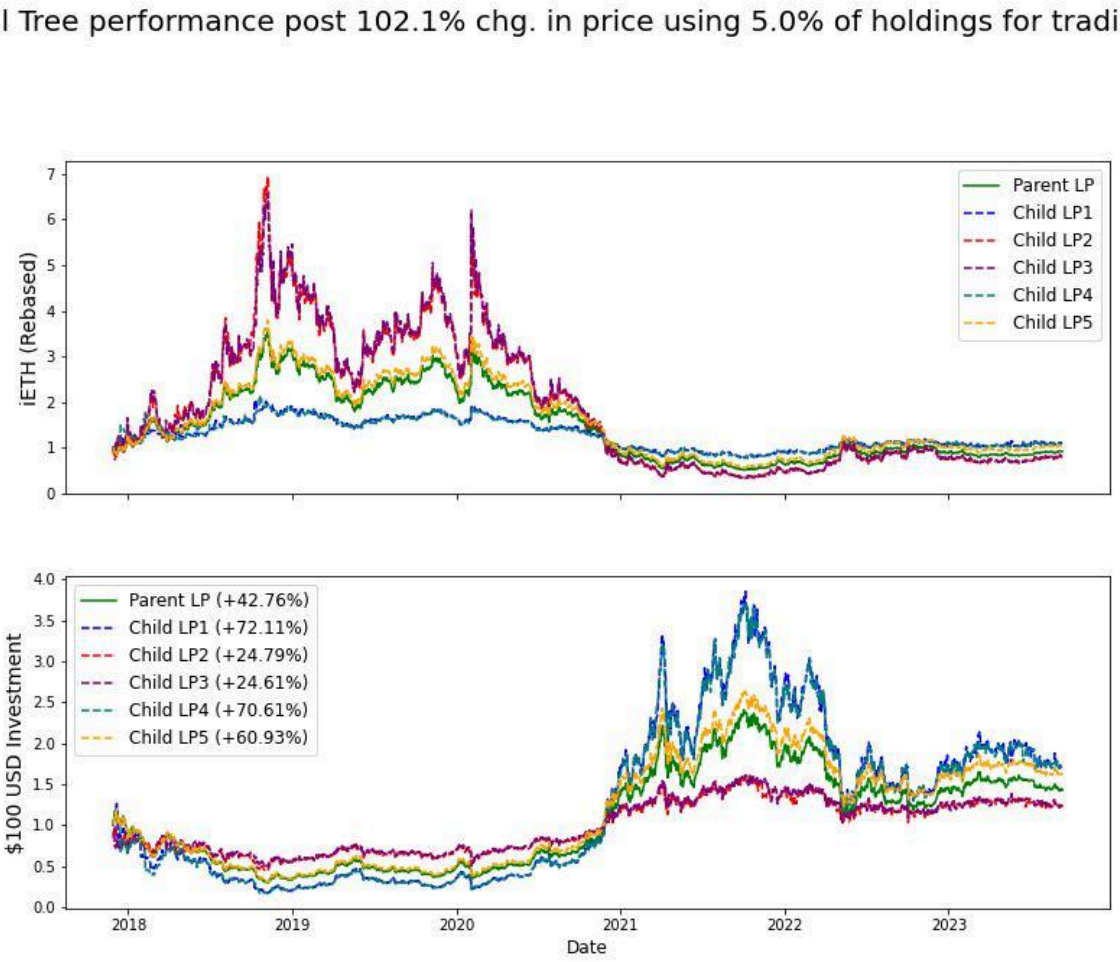

Fig 10: Full tree performance of ETH / DAI; parent LP indicated by green line is performance of standard Uniswap pool

Simulation Results

Using the Pachira simulator, we simulated the left-tree design using *ETH-USD* daily price data from January 2018 to October 2023, as shown in Fig. 11; refer to [5] for the Jupyter notebook. In the top and center images, we observe the volatile behavior of both child markets (i.e., *ETH-iDAI* and *ETH-iETH*). This volatility arises because changes in the parent LP lead to exacerbated changes in the child LPs, creating arbitrage opportunities that are seized by agents external to the Pachira system. The execution of these trades generates additional revenue streams in the child LPs through the 0.3% swap fees. The bottom image of Fig. 11 displays the fees collected from the

child LPs (shaded in red and violet regions) that would not have been realized if the left-tree structure had not been considered.

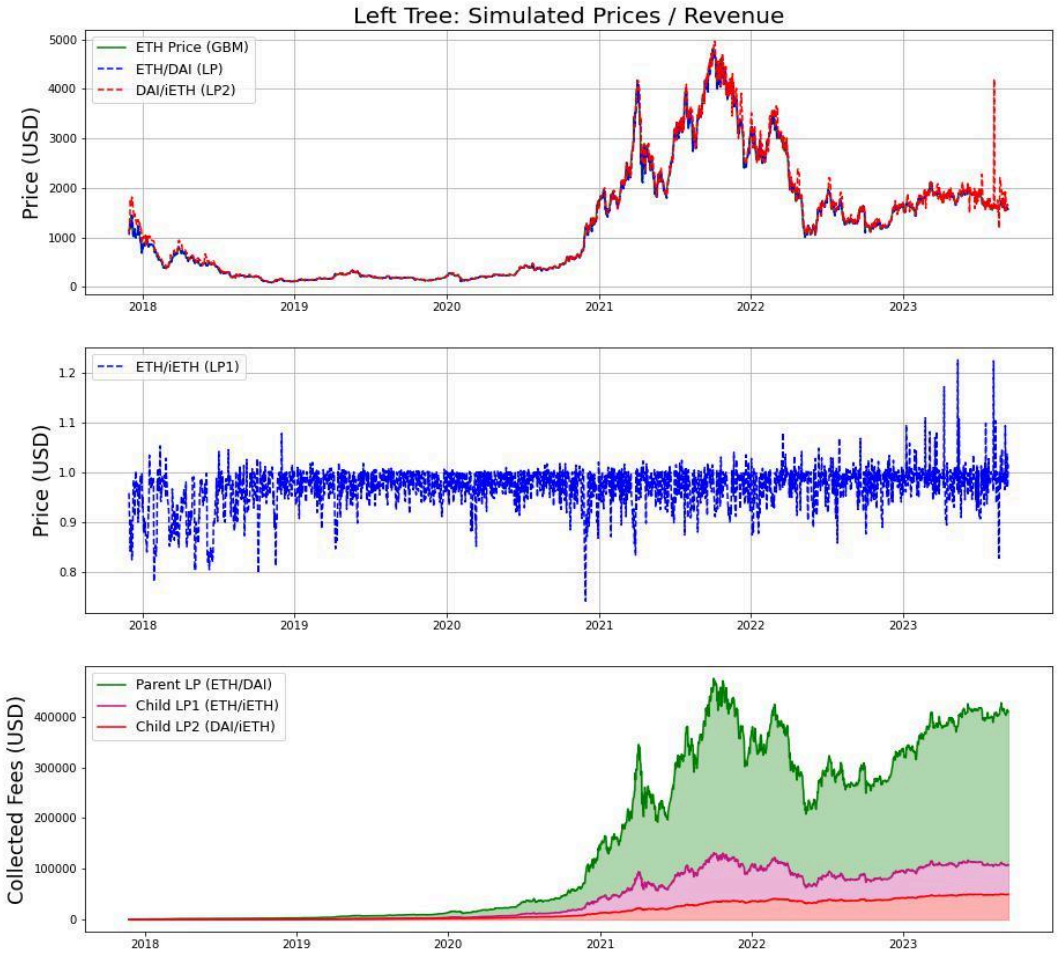

Fig 11: Simulations of Left-tree design; (TOP) *ETH-DAI* market price, simulated *ETH-DAI* price from parent, and simulated *ETH-iDAI* price from child; (CENTER) simulated *ETH-iETH* price from child; and (BOTTOM) fee revenue collected from parent and child markets

Table 1 provides tabulated statistics from the same simulation as shown in Fig. 11. Here, we observe a 35.6% revenue boost using only 7.5% indexed liquidity from the parent LP. This leveraging effect is evident in the data collected from the child LPs under the 'subtotals' heading. We observe that the total revenue / total liquidity is 80.5% and 30.6% for the *ETH-iETH* and *DAI-iETH* child LPs, respectively, compared to 19.4%

from the parent. This is because the price action in the child LPs is typically more volatile, as evidenced in the simulations.

Metric	Totals		
Revenue (LP) / LP Liquidity	19.4%		
Revenue (LP+LP1+LP2) / LP Liquidity	26.3%		
Revenue Boost (Indexed Liquidity)	35.61%		
Percentage Indexed	7.51%		
	Sub-totals		
	LP (ETH-DAI)	**LP1 (iETH-ETH)**	**LP2 (iETH-DAI)**
Liquidity	$1,559,145	$71,961	$162,118
Revenue	$302,140	$57,927	$49,673
Revenue / Liquidity	19.4%	80.5%	30.64%

Table 1: Metrics harvested from left-tree simulation using ETH and DAI (Jan 2018 to Oct 2023)

Pachira Design

In this section, we cover the components of the decentralized exchange-funded traded (DEFT) system; refer to Figs. 12-16. We will address the problems of such a system and solve them using the components of CPT indexing and Liquidity Trees discussed in previous sections. We extend those components with a governance system design, forming the complete token design of the Pachira ecosystem. This system serves three main functions, conducted in a trustless decentralized manner: (1) exchange-traded fund; (2) treasury asset issuance; and (3) a multi-asset exchange.

In Fig. 9, we present an abstracted overview of the Pachira system, which abstracts the system from the minting and burning of treasury tokens. This abstraction is also applicable to the RAI stable currency, addressing the same problem we are solving [6]. Designing such a system presents challenges. The first problem is the inefficiency issue, which we defined earlier as the Stagnant Liquidity problem. The next problem is the looping attack vector, where providers exploit the system by minting new *CHIR* treasury tokens and immediately burning them without allowing an escrow period for the value of the token in the pool to stabilize. Another issue is ensuring that the system can be completely unwound, allowing all providers to safely exit the system, in a permissionless way, at any point in time. Finally, designing a robust incentive mechanism to ensure that the value of the newly minted token will be properly shored up against the value of the other assets. These issues are addressed in the Pachira design, which can be classified into two main subsystems denoted as the expansion and reconstruction phases.

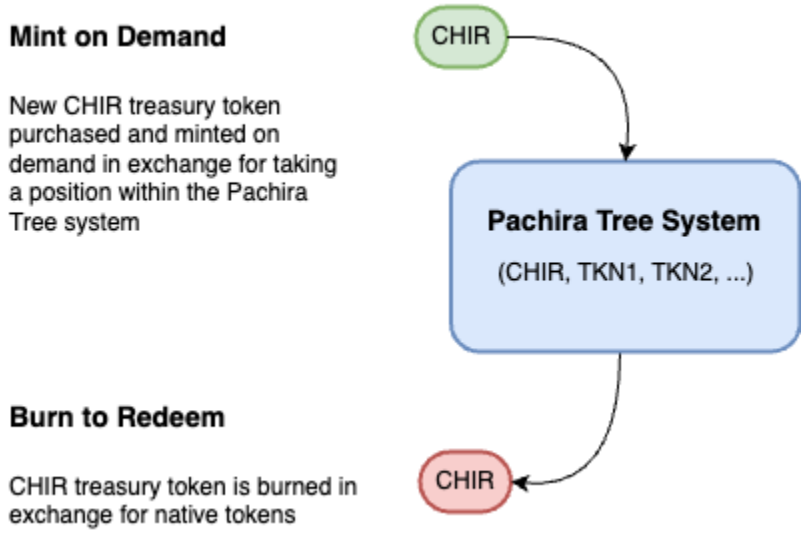

Fig 12: Abstracted overview of minting and burning process showing how Pachira handles *CHIR* treasury token issuance; *CHIR* is always fully backed by native tokens (TKN_1, TKN_2, …) within the Pachira system

Pachira System

The first problem is addressed by taking each asset, creating dual-paired CPT pools with other native assets in the system, and extending them using the liquidity tree primitive, as presented earlier. Doing this leverages existing capital and improves efficiency within the parent LP by exposing its capital through the instantiation of additional sub-markets in the child LPs using indexed liquidity from the parent. This effect creates multiple DEX trees, all integrated with each other into what we call a DEX Forest. The DEX Forest serves the function of being the expansion phase; refer to Fig. 13.

DEX Forest (Expansion Phase)

Intuitively, the expansion phase can be thought of as taking one thing, mixing it with another thing, splitting that composition into sub-buckets, and putting those sub-buckets on the market priced in terms of either one of those two things. For instance, it's akin to taking oats, mixing them with barley, and pricing the grain mix in terms of either oats or barley. As the market for the price of oats to barley changes, so does the price of the grain mix in terms of oats or barley. This is similar to what we are doing in the expansion phase, but with token assets, achieved using the Liquidity Tree primitive; see Fig. 13.

However, in addition to this expansion effect, we are also improving the efficiency of the original market in the parent LP by exposing underlying capital to new markets through the child LPs, thus addressing the Stagnant Liquidity problem outlined above. This is accomplished by integrating multiple DEX trees into what we call a DEX Forest. Hence, the DEX Forest performs two functions: (1) improving efficiency; and (2) inducing new demand underneath the treasury token, as its value is now reinforced with the already well-established assets via indexing. Both of these are achieved through the creation of new micro markets in the child LPs.

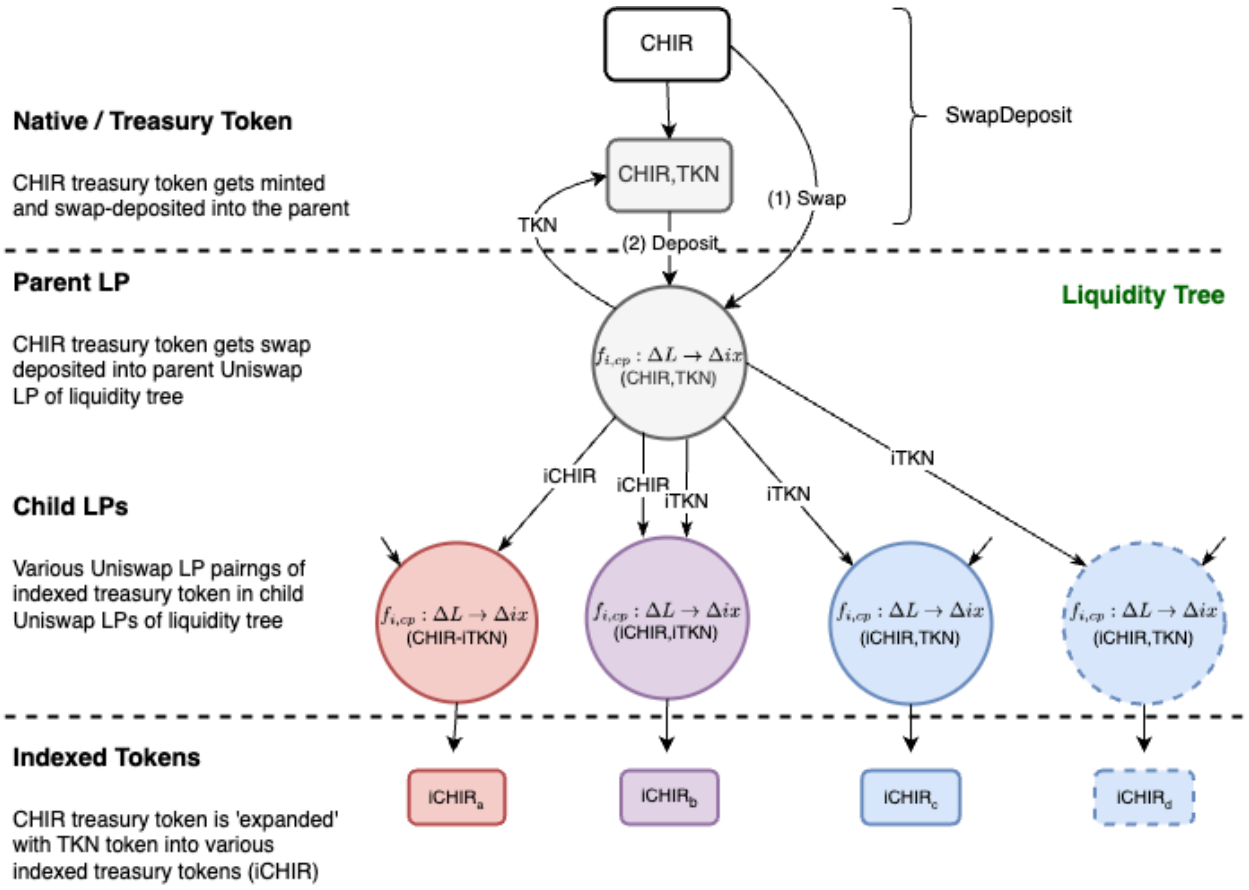

Fig 13: Expansion phase in Pachira Dex Forest design

Governance Root (Reconstruction Phase)

The governance system serves several functions. There are two main vectors that providers can take when they are looking to enter the Pachira system. The first is taking native tokens and entering directly through one of the pools within the DEX Forest, which has its advantages regarding the Stagnant Liquidity problem, as discussed

earlier. The second is through governance, which is geared for those who wish to take a position in the reserve pool. In fact, we believe this will be the path for the majority of providers. A provider enters into governance by minting index tokens, which can be done through any of the DEX trees in the forest. When this occurs, they enter into an escrow period and will receive governance tokens matching the amount of liquidity they provided into the system. This escrow period mitigates the looping attack vector as described above. During this period, governance tokens function like a float into the exchange, acting as a unit of account within the Pachira system. Governance tokens cannot be held in private wallets but are permissionless, allowing providers to take advantage of the Pachira services. When the escrow period expires, the governance token can be exchanged 1:1 for the treasury token.

In the expansion phase, we created new indexed token markets formed from a derivative of the capital within the parent LP. From each of the child LPs, we created new indexed tokens valued in either one asset or the other. While the value of each of these index tokens is close in approximation to each other, they are not fungible with each other. Hence, we combine them into composable stable pools. It is through the use of these composable stable pools that we construct the reserve tokens in the subsequent layers of the system. Hence, this phase is called the reconstruction phase, as we are reconstructing an index reserve token that is close in value to the original native token that was expanded through the DEX Forest. This phase is also known as the Governance system; see Fig. 14.

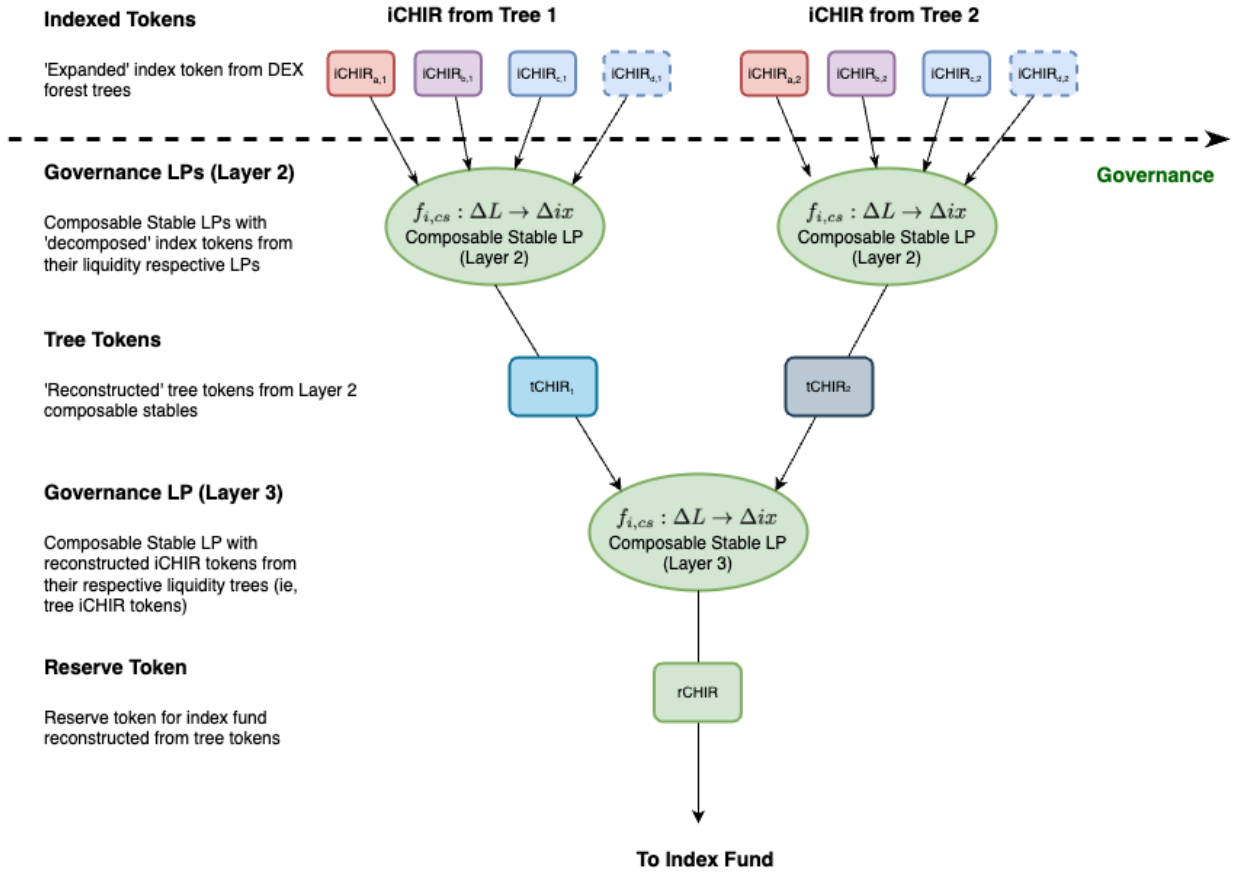

Fig 14: Reconstruction phase in Pachira design; the more closely approximated *rCHIR* is in price to the price of the original native *CHIR*, the more stable the ecosystem

Putting DEX Forest and Governance Together

Here, we integrate both the DEX Forest and Governance Root system under one unified framework, called the Pachira design; see Table 2 for token descriptions. If we consider just the reserve treasury asset, this effect of:

- Expanding it along with other well-established assets;
- Creating markets out of their composites;
- Taking the index of the composite markets, and;
- Reassembling it to something very close in value to the original native token;

is how the Pachira system trustlessly puts value underneath the *CHIR* reserve treasury asset. It is also conjectured that the sub-market activity throughout both the DEX Forest and Governance system will balance the liquidity across the trees, incentivized via the arbitrage opportunities located throughout. Hence, letting the market do the work of

putting value underneath the *CHIR* reserve treasury asset in a trustless manner. It is anticipated that the resultant effect of this will produce a reserve asset that will be closely approximated to the aggregate value of the other well-established assets (e.g., *ETH*, *BNB*, *MATIC*, etc.) within the Pachira system.

This effect is evidenced through our simulations, as shown in Figs. 17-18. In this experiment, we ran three tree simulations using *UNI-ETH*, *ETH-DAI*, and *DAI-UNI* pairings which were run independently of each other using price data from September 2020 to October 2023. Next, we fed the data from those simulations through the governance system and captured the pricing data in the reserve pool. The reserve pool is depicted by the final pool colored in orange found in both Figs. 15 and 16. As clearly seen, what we found aligned with our conjecture, which is that the reserve assets should be highly correlated to the value of the original natives. What is interesting is that we have yet to incorporate feedback mechanisms between the trees in the forest, as each tree simulation was run completely independently of each other. We conjecture that once this dependency feature is incorporated into the experiment, then the reserve asset price action will be almost identical to that of the original asset prices.

Token	Non-custodial	Placement	Description
Native (*TKN1*, *TKN2*, …)	yes	DEX Forest	Native market tokens (eg, *ETH*, *BNB*); found in parent (layer 0) LP
Treasury (*CHIR*)	yes	DEX Forest	Asset issuance treasury token (ie, *CHIR*); found in parent (layer 0) LP
Governance	no	DEX Forest / Governance	Governance token held during escrow period serving as unit of account within the Pachira system
Index	no	DEX Forest / Governance	Continuously rebasing tokens found in layer 1 and 2 LPs (eg, *iETH*, *iBNB*)
Tree	no	Governance	Continuously rebasing tokens found in layer 3 LP; aggregate of each individual tree activity (eg, *tETH*, *tBNB*)
Native Reserve	no	Governance	Continuously rebasing tokens found in layer 4 LP; these token trade very closely in price to Native tokens
Reserve	no	Governance	Index token of the entire system serving as ETF; LP token of reserve (CWPT) pool

Table 2: Description of each of the token's role, placement, and function within the Pachira system

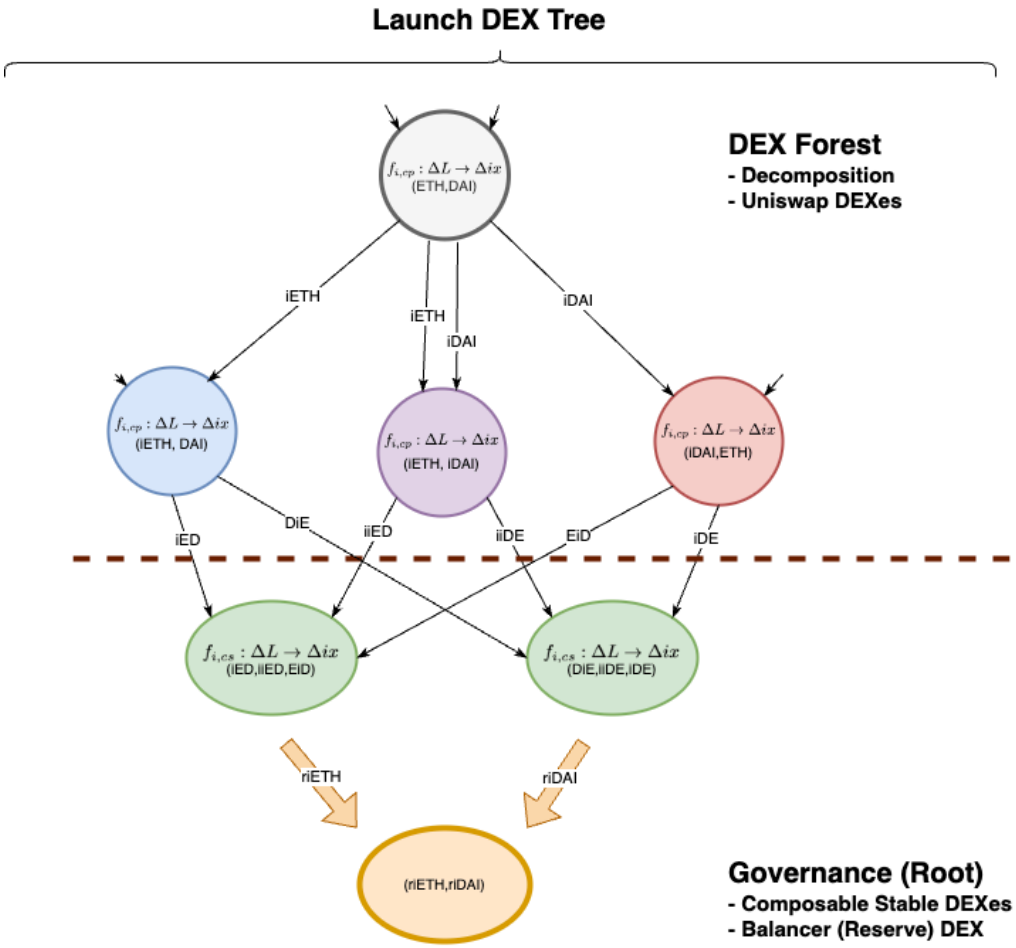

Fig 15: Putting expansion (Fig. 13) and reconstruction (Fig. 14) phases together into a single-asset Pachira system

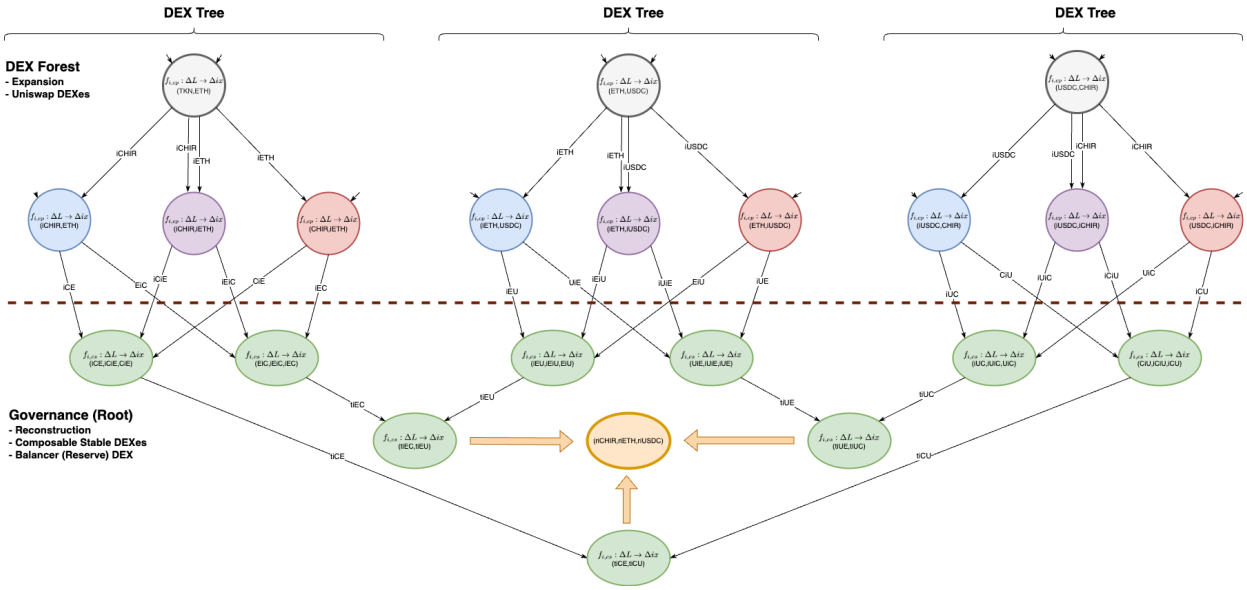

Fig 16: Putting expansion (Fig. 13) and reconstruction (Fig. 14) phases together into a three-asset Pachira system. Multitree systems introduces deep governance for trustlessly balancing *CHIR* treasury token throughout the trees in the DEX forest ecosystem

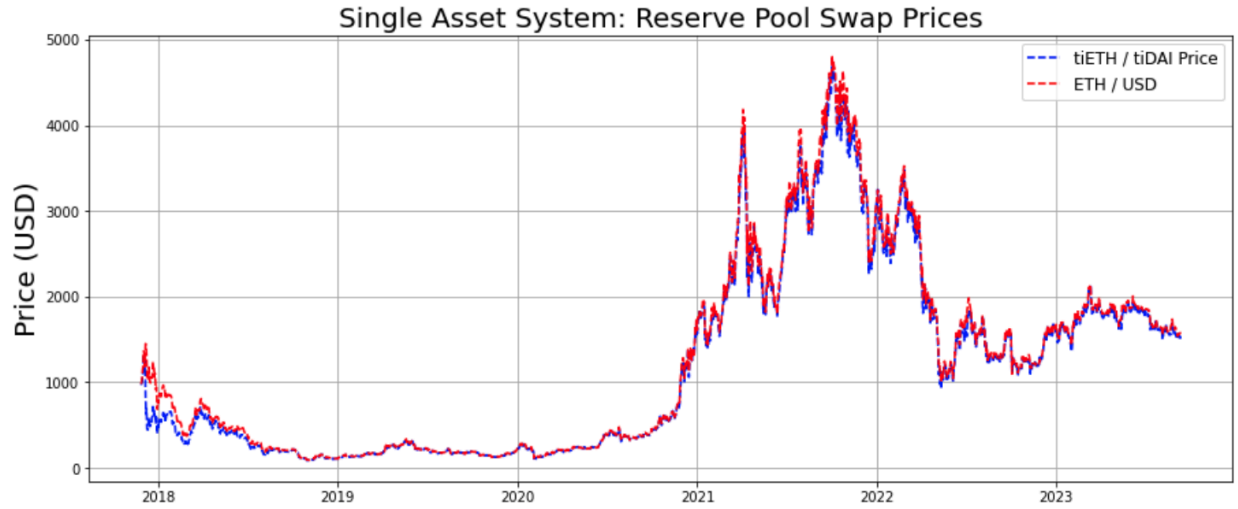

Fig 17: Simulated swap prices (ie, tiETH / tiDAI) compared to native market prices

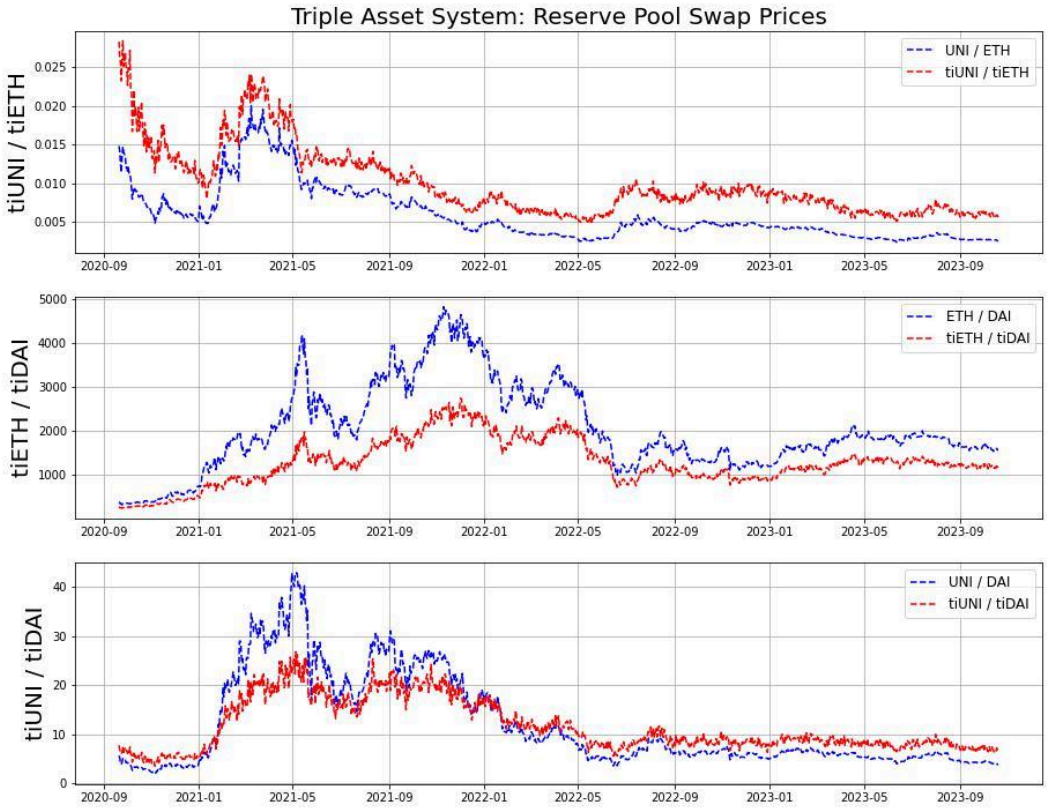

Fig 18: Simulated swap prices (ie, tiUNI / tiETH, tiETH / tiDAI, and tiUNI / tiDAI) compared to native market prices; because simulations were run independent of each other, these trees are slightly out of balance and come into equilibrium when tiTKN = TKN

CHIR Token Summary

In this paper, we introduced a new DeFi primitive, termed the Liquidity Tree, which addresses an inefficiency in DeFi that we have called the Stagnant Liquidity problem. Liquidity trees work by taking stagnated liquidity within an LP and indexing it into a continuous rebasing token (CRIT). CRITs are akin to fully collateralized loans that can be fully redeemed for the value of the underlying LP token. These tokens are placed into markets called child LPs along with one of the two residing assets in the parent LP. We show how this liquidity tree structure addresses a capital inefficiency issue highlighted by the Stagnant Liquidity problem.

CHIR serves as the treasury token in the Pachira system and resides with other native tokens (e.g., *ETH*, *BNB*, etc.) in a collection of Liquidity Trees called a Liquidity Forest; see Fig. 16. The Liquidity Forest earns additional yield off its collection of tokens (i.e., treasury and native tokens), which gets expanded into CRITs. The governance system then works to reconstruct the value of these CRITs into a reserve pool of tokens that trade close in value to the original native tokens. The reserve pool is akin to a decentralized yield-bearing ETF and serves as a way for LP providers to trustlessly invest in a basket of assets and earn additional yield.

Aside from being a decentralized yield-bearing ETF, the Pachira system also serves two other functions: as a multi-asset decentralized exchange using the governance token as its unit of account and as a treasury asset issuance system using *CHIR*, which is fully backed by the other native tokens within the Pachira system. All these functions are handled at the contract level, making this liquidity management system immutable and fully trustless. Thus, it does not rely on involvement from other third-party stakeholders. Due to these attributes, there are no other comparables to Pachira, as it utilizes a novel DeFi primitive, which we have termed Liquidity Trees, to address the Stagnant Liquidity inefficiency problem in a way that no other entity has done before.

References

[1] I. Moore**, "**How to Handle Uniswap Withdrawals like an OG**",** *Coinmonks***,** Oct 5, 2023.
[2] I**.** Moore**, "**The Uniswap Indexing Problem**",** *The Data Driven Investor***,** Oct 8, 2023.
[3] C. Doge, Liquidity Trees: Tech Docs.
[4] I. Moore, "How to Simulate a Liquidity Pool for Decentralized Finance", *Medium Article***,** Mar 10, 2023.
[5] Syscoin Platform, "Pachira Tree Simulator" [Source Code]; Github repository, 2023, online: https://github.com/syscoin/pachira-simulator
[6] V. Buterin, "Two thought experiments to evaluate automated stablecoins", May 25, 2022.